Расширенный курс языка C++

К.А. Кутасин, Л. М. Львова

Домашнее задание 3

Требования к программам

1. Для решения каждой задачи предлагается реализовать класс на языке программирования C++.
2. Класс должен называться в точности как указано в задании.
3. Класс должен реализовывать все методы указанные в задании.
4. Исходный код класса должен содержаться в двух файлах: `classname.h`, `classname.cpp`, где `classname` – указанное в задании имя класса.
5. Исходный код должен быть оформлен согласно набору правил и рекомендаций стандарта кодирования GNU.
6. Использовать реализации аналогичных классов из стандартной библиотеки языка в качестве подлежащей реализации запрещено.
7. В случае возникновения вопросов по условию, можно использовать сайт `cppreference.com`. Поведение класса, описанное на этом сайте считается верным.
8. Использовать примеры реализации класса из интернета, в частности, с сайта `cppreference.com` запрещено. Использование искусственного интеллекта запрещено.
9. Проверка задания осуществляется путем использования реализованного класса в тестовой программе. Тестовая программа использует все указанные в задании методы и никакие другие.
10. Если тестовая программа компилируется и корректно работает с приведенной реализацией класса, а реализация соответствует приведенным требованиям, то задание считается выполненным.
11. Если тестовая программа не компилируется или некорректно работает с приведенной реализацией класса, то задание считается не выполненным.
12. Если реализация класса не соответствует приведенным требованиям, то задание считается не выполненным.

Задачи

1. Написать класс **string**, реализующий работу со строкой. Класс должен поддерживать два вида инициализации: по символу и числу повторений, а так же по строковому литералу. Конструктор по умолчанию должен создавать валидный экземпляр класса. Класс должен поддерживать все указанные операции. Класс не должен содержать публичных методов за исключением указанных, все вспомогательные методы должны быть приватными. Объявление класса приведено ниже:

```
class string
{
public:
    string ();
    string (char val, size_t count);
    string (const char *str);
    ~string () { delete [] data; }

    string (const string &val);
    string &operator= (const string &val);

    string (string &&val);
    string &operator= (string &&val);
```

```
    string operator+ (const string &val);
    string &operator+= (const string &val);

    bool operator== (const string &val);
    bool operator< (const string &val);

    char operator[] (size_t i) const;

    // Вернуть true, если строка содержит подстроку (substr)
    bool contains (const string &substr) const;

    // Заменить подстроку (what) подстрокой (val)
    // Вернуть true, если замена произошла
    bool replace (const string &what, const string &val);

    const char *c_str () const { return data; }
    size_t size () const { return _size; }
    // true если строка пустая
    bool empty () const;

    // выполнить конкатенацию со строкой rhs
    // вернуть ссылку на текущий объект
    string &append (const string &rhs);

    // удалить подстроку от индекса start до индекса end
    // вернуть ссылку на текущий объект
    // NOTE: индекс end -- не включительно!
    // NOTE: erase (0, size) удаляет всю строку
    // NOTE: erase (0, 0) не удаляет ничего
    string &erase (size_t start, size_t end);

    // вернуть подстроку от индекса start до индекса end
    // NOTE: индекс end -- не включительно!
    // NOTE: substr (0, size) возвращает копию строки
    // NOTE: substr (0, 0) возвращает пустую строку
    string substr (size_t start, size_t end) const;

    size_t find (const string &substr) const;
    size_t rfind (const string &substr) const;

private:
    char *data{};
    size_t _size{};
};
```

2. Написать класс **vector_string**, реализующий работу с массивом строк. Строка представлена классом `string` из задания 1. Конструктор по умолчанию должен создавать валидный экземпляр класса. Класс должен поддерживать все указанные операции. Класс не должен содержать публичных методов за исключением указанных, все вспомогательные методы должны быть приватными. Объявление класса приведено ниже:

```
class vector_string
{
public:
    vector_string ();
    // Инициализировать массив копиями val, в количестве count
    vector_string (const string &val, size_t count);
    ~vector_string ();

    vector_string (const vector_string &val);
    vector_string &operator= (const vector_string &val);

    vector_string (vector_string &&val);
    vector_string &operator= (vector_string &&val);
```

```
    vector_string operator+ (const vector_string &val);
    vector_string &operator+= (const vector_string &val);

    const string &operator[] (size_t i) const;
    string &operator[] (size_t i);

    size_t size () const { return _size; }
    // true если массив пустой
    bool empty () const;

    // добавить копию строки str в конец массива
    // вернуть ссылку на текущий объект
    vector_string &push_back (string &&str);

    // добавить копию строки str в начало массива
    // вернуть ссылку на текущий объект
    vector_string &push_front (string &&str);

    // выполнить конкатенацию с массивом rhs
    // вернуть ссылку на текущий объект
    vector_string &append (vector_string &&rhs);
    vector_string &append (const vector_string &rhs);

    // удалить подмассив от индекса start до индекса end
    // вернуть ссылку на текущий объект
    // NOTE: индекс end -- не включительно!
    // NOTE: erase (0, size) удаляет весь подмассив
    // NOTE: erase (0, 0) не удаляет ничего
    vector_string &erase (size_t start, size_t end);

    // индекс первого вхождения строки str
    // сравнение на точное равенство
    // вернуть size если подстроки нет
    size_t find (const string &str) const;

    // индекс последнего вхождения строки str
    // сравнение на точное равенство
    // вернуть size если подстроки нет
    size_t rfind (const string &str) const;

    // заменить подстроку what подстрокой val во всех элементах
    // вернуть ссылку на текущий объект
    vector_string &replace (const string &what, const string &val);

    // вернуть подмассив от индекса start до индекса end
    // NOTE: индекс end -- не включительно!
    // NOTE: substr (0, size) возвращает копию массива
    // NOTE: substr (0, 0) возвращает пустой массив
    vector_string subvector (size_t start, size_t end) const;

private:
    string *data{};
    size_t _size{};
};
```

3. Написать класс **tree_map_string**, реализующий работу с простым бинарным деревом поиска (ключ – строка, значение – другая строка, сравнение – `operator<`). Строка представлена классом `string` из задания 1. Класс должен поддерживать все указанные операции. Объявление класса приведено ниже:

```
class tree_map_string
{
public:
    tree_map_string ();
    ~tree_map_string ();

    tree_map_string (const tree_map_string &val);
    tree_map_string &operator= (const tree_map_string &val);

    tree_map_string (tree_map_string &&val);
    tree_map_string &operator= (tree_map_string &&val);

    size_t size () const { return _size; }
    bool empty () const;

    // Вернуть значение по ключу (it)
    // Бросить исключение, если такого ключа нет
    const string &at (const string &it) const;
    string &at (const string &it);

    // Вернуть значение по ключу (it)
    // Создать по умолчанию и вернуть, если такого ключа нет
    string &operator[] (const string &it);

    // Вставить значение (what) по ключу (it)
    // Вернуть true, если вставка прошла успешно
    // Вернуть false, если такой ключ уже есть
    bool insert (const string &it, const string &what);
    bool insert (string &&it, string &&what);

    // Удалить значение по ключу (it)
    // Вернуть true, если удаление прошло успешно
    // Вернуть false, если такого ключа нет
    bool erase (const string &it);

    // Заменить подстроку (what) на подстроку (val) во всех элементах
    // Вернуть количество измененнных элементов
    size_t replace (const string &what, const string &val);

    // Вырезать (переместить все узлы) поддерево с корнем в узле с ключом (it)
    // Если такого ключа нет, создать пустое дерево
    tree_map_string cut_subtree (const string &it) const;

    // Вернуть количество значений с ключом строго меньшим заданного (it)
    size_t count_lt (const string &val) const;

private:
    tree_node *root{};
    size_t _size{};
};
```

4. Написать класс **hash_map_string**, реализующий работу с хэш-таблицей (ключ – строка, значение – другая строка, хэш – первый символ строки). Строка представлена классом `string` из задания 1. Класс должен поддерживать все указанные операции. Объявление класса приведено ниже:

```
class hash_map_string
{
public:
    hash_map_string ();
    ~hash_map_string ();

    hash_map_string (const hash_map_string &val);
    hash_map_string &operator= (const hash_map_string &val);

    hash_map_string (hash_map_string &&val);
    hash_map_string &operator= (hash_map_string &&val);

    size_t size () const { return _size; }
    bool empty () const;

    // Вернуть значение по ключу (it)
    // Бросить исключение, если такого ключа нет
    const string &at (const string &it) const;
    string &at (const string &it);

    // Вернуть значение по ключу (it)
    // Создать по умолчанию и вернуть, если такого ключа нет
    string &operator[] (const string &it);

    // Вставить значение (what) по ключу (it)
    // Вернуть true, если вставка прошла успешно
    // Вернуть false, если такой ключ уже есть
    bool insert (const string &it, const string &what);
    bool insert (string &&it, string &&what);

    // Удалить значение по ключу (it)
    // Вернуть true, если удаление прошло успешно
    // Вернуть false, если такого ключа нет
    bool erase (const string &it);

    // Заменить подстроку (what) на подстроку (val) во всех элементах
    // Вернуть количество измененнных элементов
    size_t replace (const string &what, const string &val);

    // Вырезать (переместить все узлы) все узлы с хэшем (val)
    // Если такого ключа нет, создать пустой объект
    hash_map_string cut_subhash (unsigned char val) const;

    // Вернуть количество значений с ключом строго меньшим заданного (it)
    size_t count_lt (const string &val) const;

private:
    // Отдельный объект под каждое значение хэша (unsigned char)
    // Unsigned char принимает значения от 0 до 255
    hash_node head[256];
    size_t _size{};
};
```